

20 Formulas That Explain Machine Learning

A Gentle Walk Through Pattern Recognition and Machine Learning

Dr. Ye Kyaw Thu

Language Understanding Lab, AI Mario, Myanmar

Visiting Professor, Core-AI Research Team, NECTEC, Thailand

Introduction

What This Course Is

- This is a **gentle tour** of one famous book: *Pattern Recognition and Machine Learning* by Christopher Bishop.
- The book has **738 pages** and **14 chapters**.
- We will compress it into **20 formulas**.
- Every formula will be explained with:
 - A plain-English translation
 - Every symbol defined
 - A tiny numerical example you can do by hand
 - Why it matters in real life
- **No advanced math needed.** If you know what a sum is and what a fraction is, you can follow along.

The 20 Formulas We Will Learn

Part 1 — Foundation (4)

1. Sum rule
2. Bayes' theorem
3. Gaussian distribution
4. Expectation

Part 2 — Regression (3)

5. Linear model
6. Sum-of-squares error
7. Normal equations

Part 3 — Classification (3)

8. Logistic sigmoid
9. Logistic regression
10. Cross-entropy loss

Part 4 — Nonlinear & Kernels (3)

11. Kernel function
12. SVM margin
13. Gaussian process

Part 5 — Hidden Structure (4)

14. Gaussian mixture
15. KL divergence
16. EM algorithm
17. PCA

Part 6 — Sequences (3)

18. HMM joint
19. Forward algorithm
20. Kalman filter

How to Read a Formula

Take this simple formula:

$$y = 3x + 2$$

- **Left side** (y): the thing we want to compute.
- **Right side** ($3x + 2$): the recipe.
- **Input** (x): the number we plug in.
- **Numbers** ($3, 2$): the *parameters* — the knobs we tune.

Tiny example (try it by hand!):

If $x = 5$, then $y = 3 \times 5 + 2 = 17$.

That's all machine learning formulas are: **a recipe**.

The Two Most Important Words

Probability

- A number between 0 and 1.
- 0 = impossible; 1 = certain.
- Example: “ $p(\text{rain tomorrow}) = 0.3$ ” means 30% chance.

Random variable

- A value we don't know yet.
- Example: X = the outcome of a dice roll.
- We write $p(X = x)$ for the probability that X takes value x .

Machine learning is just the art of **guessing random variables** and **updating our guesses** when we see data.

Formula 1: The Sum Rule

Formula 1 — The Sum Rule

$$p(X) = \sum_Y p(X, Y)$$

“To get the probability of X , add up the probabilities of X for every possible value of Y .”

X	the variable we care about
Y	another variable we want to ignore
$p(X, Y)$	joint probability of both X and Y
$\sum_Y$	sum over every possible value of Y

Sum Rule — Toy Example

Tiny example (try it by hand!):

We roll two dice: X (red die) and Y (blue die).

Joint probabilities:

	$Y = 1$	$Y = 2$	$Y = 3$
$X = 1$	0.1	0.05	0.02
$X = 2$	0.08	0.15	0.10
$X = 3$	0.05	0.20	0.25

What is $p(X = 2)$? Add the whole row for $X = 2$:

$$p(X = 2) = 0.08 + 0.15 + 0.10 = 0.33$$

Summing out Y gives the **marginal** of X .

Sum Rule — Why It Matters

Why this matters: Whenever you have a table with 2 variables and you only want to know about one of them, you sum out the other. This is the foundation of Bayesian inference.

- Spam filters: $p(\text{email is spam}) = \text{sum over all possible words.}$
- Medical tests: $p(\text{disease}) = \text{sum over all symptom combinations.}$
- Search engines: $p(\text{query}) = \text{sum over all possible users.}$

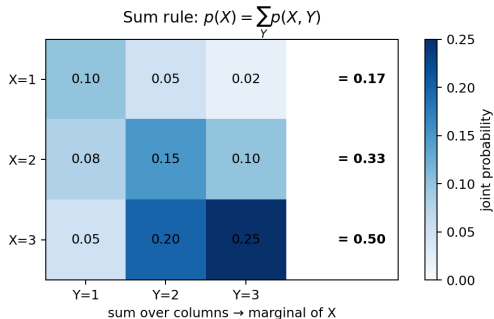
notebook: density-estimation.ipynb

Formula 1 – Sum Rule: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt

joint = np.array([[.10,.05,.02],[.08,.15,.10],[.05,.20,.25]])
marginal_x = joint.sum(axis=1)
fig, ax = plt.subplots(figsize=(6.4, 3.6))
im = ax.imshow(joint, vmin=0, vmax=joint.max(), cmap='Blues')
for i in range(3):
    for j in range(3): ax.text(j, i, f'{joint[i,j]:.2f}', ha='center', va='center')
for i, v in enumerate(marginal_x): ax.text(3.15, i, f'{v:.2f}', va='center', fontweight='bold')
ax.set_xticks(range(3), ['Y=1', 'Y=2', 'Y=3']); ax.set_yticks(range(3), ['X=1', 'X=2', 'X=3'])
ax.set_xlim(-.5, 3.9); ax.set_title(r'Sum rule:  $\sum_Y p(X,Y)$ ')
ax.set_xlabel('sum over columns → marginal of X')
fig.colorbar(im, ax=ax, label='joint probability')
fig.tight_layout(); fig.savefig('../fig/formula01_sum_rule.png', dpi=180); plt.close(fig)
```



Read across the row: summing over Y removes Y and leaves the marginal of X .

notebook: `density-estimation.ipynb`

Notebook idea: marginalization / density estimation.

Formula 2: Bayes' Theorem

Formula 2 — Bayes' Theorem

$$p(H | D) = \frac{p(D | H) p(H)}{p(D)}$$

“My updated belief about H equals how well H explains the data D , times my prior belief about H , divided by how likely the data is overall.”

H	hypothesis (something we believe)
D	data (something we observed)
$p(H)$	prior — belief before data
$p(D H)$	likelihood — how well H explains D
$p(H D)$	posterior — belief after data
$p(D)$	evidence — normaliser

Bayes' Theorem — Toy Example (The Cancer Test)

Tiny example (try it by hand!):

Setup. 1000 people are tested.

- 1 in 100 has cancer $\Rightarrow$ 10 sick, 990 healthy.
- Test is 99% accurate: 99% true positive, 1% false positive.

Question. A person tests positive. Do they have cancer?

- Sick and positive: $10 \times 0.99 = 9.9$ people.
- Healthy but positive: $990 \times 0.01 = 9.9$ people.
- Total positives: $9.9 + 9.9 = 19.8$.

$$p(\text{cancer} \mid +) = \frac{9.9}{19.8} \approx 0.5$$

Even with a 99% accurate test, a positive result only gives a 50% chance of cancer — because cancer is rare.

Bayes' Theorem — Why It Matters

Why this matters: Bayes' theorem is **the** equation of machine learning. Every probabilistic model you will ever build uses it.

- Spam filter: $p(\text{spam} \mid \text{words})$
- Medical diagnosis: $p(\text{disease} \mid \text{symptoms})$
- Speech recognition: $p(\text{words} \mid \text{sound})$
- ChatGPT-style models: $p(\text{next word} \mid \text{previous words})$

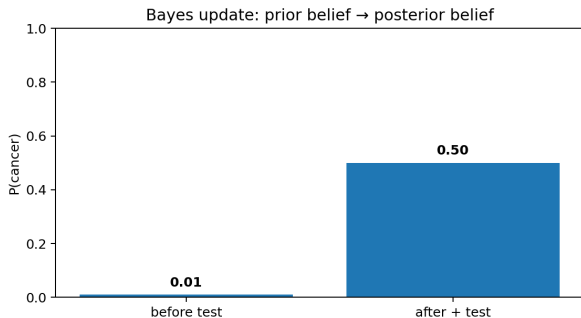
notebook: introduction.ipynb

Formula 2 – Bayes' Theorem: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt

prior = 0.01
sensitivity = 0.99
false_positive = 0.01
posterior = sensitivity*prior/(sensitivity*prior +
    false_positive*(1-prior))
vals = [prior, posterior]
fig, ax = plt.subplots(figsize=(6.4,3.6))
ax.bar(['before test', 'after + test'], vals)
ax.set_ylim(0, 1); ax.set_ylabel('P(cancer)'); ax.set_title("
    Bayes update: prior belief → posterior belief")
for i,v in enumerate(vals): ax.text(i, v+.03, f'{v:.2f}', ha=
    'center', fontweight='bold')
fig.tight_layout(); fig.savefig('./fig/
    formula02_bayes_theorem.png', dpi=180); plt.close(fig
)
```



The posterior changes because prior belief and the likelihood of the observed data are combined.

notebook: bayes-normal.ipynb

Notebook reference: Gaussian Bayesian updating.

Formula 3: The Gaussian Distribution

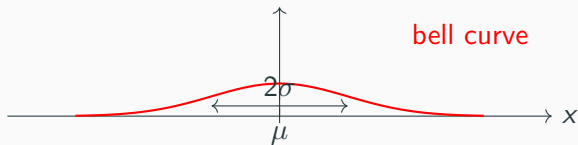
Formula 3 — The Gaussian (Normal) Distribution

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

“The bell curve. The probability of seeing x is highest when x is close to the mean μ , and drops off smoothly as we move away.”

x	the value we observed
μ	the mean (centre of the bell)
σ^2	the variance (spread of the bell)
σ	the standard deviation ($\sqrt{\sigma^2}$)
$\exp(\cdot)$	the exponential function ($e^{\cdot}$, roughly 2.718)
π	the number 3.14159... (only used to make the area = 1)

The Bell Curve in Pictures



- **Tall and narrow:** small variance (all values cluster).
- **Short and wide:** large variance (values spread out).
- About **68%** of values lie within $\mu \pm \sigma$.
- About **95%** lie within $\mu \pm 2\sigma$.

Gaussian — Toy Example

Tiny example (try it by hand!):

Heights of adult men: $\mu = 175$ cm, $\sigma = 10$ cm.

What fraction of men are between 165 and 185 cm?

- $165 = \mu - \sigma$, $185 = \mu + \sigma$.
- The interval $\mu \pm \sigma$ contains about **68%**.

What is the most common height? **175 cm** (the mean).

What about 200 cm? Extremely rare — about 6 cm (0.6σ) above the 95% range.

The Gaussian is the world's favourite distribution because **averages of many random effects look Gaussian** (the Central Limit Theorem).

Why this matters: Whenever you assume “noise” in your data, you almost always assume it is Gaussian. This makes the math tractable.

- Noise in sensors, cameras, microphones.
- Stock returns (approximately).
- Measurement errors in every scientific experiment.
- The basis for Kalman filters, Gaussian processes, probabilistic PCA.

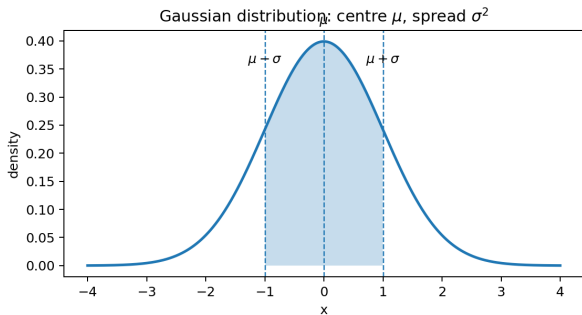
notebook: bayes-normal.ipynb

Formula 3 – The Gaussian Distribution: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-4,4,600); mu, sigma = 0, 1
p = np.exp(-0.5*((x-mu)/sigma)**2)/(np.sqrt(2*np.pi)*sigma)
fig, ax = plt.subplots(figsize=(6.4,3.6)); ax.plot(x,p,
    linewidth=2)
for k in (-1,0,1): ax.axvline(mu+k*sigma, linestyle='--',
    linewidth=1)
ax.text(-1,.36,r'\mu-\sigma',ha='center'); ax.text(0,.43,r'
    \mu',ha='center'); ax.text(1,.36,r'\mu+\sigma',ha
    ='center')
ax.fill_between(x[(x>=-1)&(x<=1)], p[(x>=-1)&(x<=1)], alpha
    =.25)
ax.set_title(r'Gaussian distribution: centre $\mu$, spread $\sigma^2$');
ax.set_xlabel('x'); ax.set_ylabel('density')
fig.tight_layout(); fig.savefig('../fig/formula03_gaussian.
    png', dpi=180); plt.close(fig)
```



The mean moves the centre; the variance controls how concentrated or spread out the density is.

notebook: `bayes-normal.ipynb`

Notebook reference: normal model and Gaussian updating.

Formula 4: Expectation

Formula 4 — Expectation (The Average)

$$\mathbb{E}[f(x)] = \sum_x p(x) f(x) \quad \text{or} \quad \mathbb{E}[f(x)] = \int p(x) f(x) dx$$

“The average value of $f(x)$, where the average is weighted by how likely each x is.”

$f(x)$ any function of the random variable x

$p(x)$ probability of x

$\sum$ sum over all possible x (discrete case)

$\int$ integral over all x (continuous case)

$\mathbb{E}[\cdot]$ the expectation (weighted average)

Expectation — Toy Example

Tiny example (try it by hand!):

You roll a fair die. The number x is random, equally likely to be 1, 2, 3, 4, 5, 6.

$$\mathbb{E}[x] = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = 3.5$$

You don't gain 3.5 on a single roll — but if you play 1000 times, your average is close to 3.5.

Now suppose $f(x) = x^2$ (you win the square of the roll):

$$\mathbb{E}[x^2] = \frac{1}{6}(1 + 4 + 9 + 16 + 25 + 36) = \frac{91}{6} \approx 15.2$$

Expectation is **the long-run average** over many trials.

Expectation — Why It Matters

Why this matters: In machine learning, we usually want to **minimise the expected loss** (average error over many possible worlds).

- Expected spam-filter error rate.
- Expected profit in trading.
- Expected risk in medicine.
- When we “train” a model, we are computing expectations over the data.

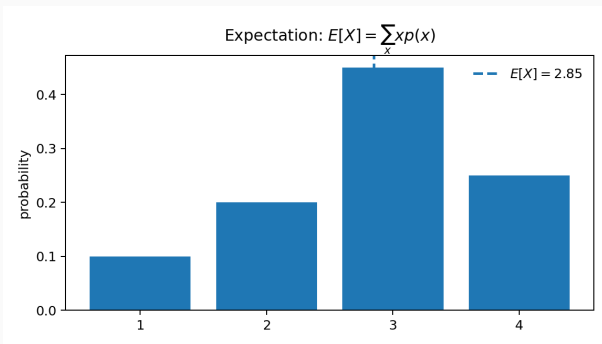
notebook: introduction.ipynb

Formula 4 – Expectation: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,3,4]); p = np.array([.10,.20,.45,.25]); ex
    = (x*p).sum()
fig, ax = plt.subplots(figsize=(6.4,3.6)); ax.bar(x,p)
ax.axvline(ex, linestyle='--', linewidth=2, label=rf'$E[X]={
    ex:.2f}$')
ax.set_xticks(x); ax.set_ylabel('probability'); ax.set_title(
    r'Expectation: $E[X]=\sum_x x p(x)$'); ax.legend(
    frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula04_expectation
    .png', dpi=180); plt.close(fig)
```



Expectation is the probability-weighted centre: values with more probability pull the average toward them.

A standalone teaching example for the expectation formula.

Formula 5: The Linear Model

Formula 5 — The Linear Model

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_Dx_D$$

“Our prediction y is a weighted sum of the input features. Each feature x_i is multiplied by a weight w_i , and we add a constant w_0 (the bias).”

y	our prediction (a single number)
$x_1, \dots, x_D$	the input features (numbers we measured)
w_0	the bias (baseline value when all $x_i = 0$)
$w_1, \dots, w_D$	the weights (importance of each feature)
D	number of features

Linear Model — Toy Example (House Prices)

Tiny example (try it by hand!):

Predict price (in \$1000s) from 2 features:

- x_1 = size in 100s of square feet
- x_2 = number of bedrooms

Our model: $y = 20 + 30 x_1 + 5 x_2$

- $w_0 = 20$: base price. $w_1 = 30$: each 100 sq ft adds \$30k.
- $w_2 = 5$: each bedroom adds \$5k.

Predict for a 1500 sq ft, 3-bedroom house:

$$y = 20 + 30 \times 15 + 5 \times 3 = 20 + 450 + 15 = 485.$$

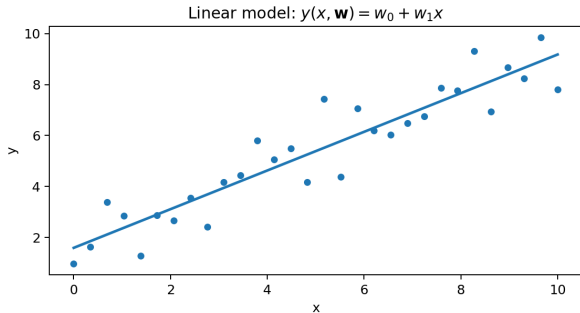
So **\$485,000**.

notebook: `linear-models-for-regression.ipynb`

Formula 5 – The Linear Model: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)
x = np.linspace(0, 10, 30); y = 1.5 + .8*x + rng.normal(0,.8,
    len(x))
X = np.column_stack([np.ones_like(x), x]); w = np.linalg.
    lstsq(X,y,rcond=None)[0]
xx = np.linspace(0,10,200)
fig, ax = plt.subplots(figsize=(6.4,3.6)); ax.scatter(x,y,s
    =18); ax.plot(xx,w[0]+w[1]*xx,linewidth=2)
ax.set_title(r'Linear model:  $y(x, \mathbf{w}) = w_0 + w_1x$ '); ax
    .set_xlabel('x'); ax.set_ylabel('y')
fig.tight_layout(); fig.savefig('../fig/
    formula05_linear_model.png', dpi=180); plt.close(fig)
```



The model is linear in the weights w even when we later replace x by richer basis functions.

notebook: `linear-models-for-regression.ipynb`

Notebook reference: linear regression with basis functions.

Formula 6: Sum-of-Squares Error

Formula 6 — Sum-of-Squares Error

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2$$

“For every training example, take the difference between our prediction and the true value, square it, and add them all up.”

N	number of training examples
$\mathbf{x}_n$	the n -th input
t_n	the n -th true target
$y(\mathbf{x}_n, \mathbf{w})$	our prediction for example n
$E(\mathbf{w})$	total error (a single number)
$\frac{1}{2}$	factor of 1/2 — makes later math cleaner

Why Squared Error? Why Not Just Absolute?

- **Square** makes all errors positive (no cancellation).
- **Square** punishes big errors more than small ones — good.
- It is **smooth**, so we can differentiate it (calculus!).
- It corresponds to assuming the noise is **Gaussian**.

Imagine drawing vertical lines from each data point to the curve. The error E is the sum of the **squared lengths** of those lines. We want to make them as short as possible.

Tiny example (try it by hand!):

Predictions = (2, 4, 6). Truth = (3, 3, 8).

Errors = (-1, 1, -2). Squared = (1, 1, 4). Sum = 6.

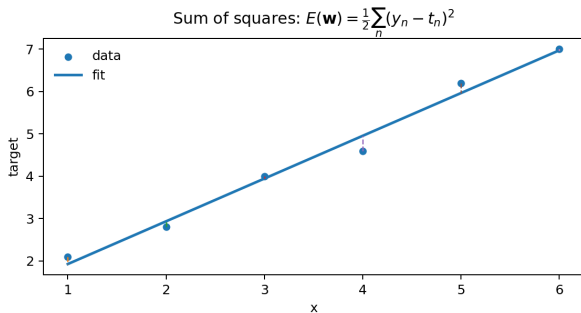
So $E = 6/2 = 3$.

notebook: ml-vs-map.ipynb

Formula 6 – Sum-of-Squares Error: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)
x = np.arange(1,7); y = np.array([2.1,2.8,4.0,4.6,6.2,7.0]);
w=np.polyfit(x,y,1); pred=np.polyval(w,x); residual=y
-pred
fig, ax = plt.subplots(figsize=(6.4,3.6)); ax.scatter(x,y,s
=22,label='data'); ax.plot(x,pred,linewidth=2,label='
fit')
for xi,yi,pi in zip(x,y,pred): ax.plot([xi,xi],[yi,pi],
linestyle='--',linewidth=1)
ax.set_title(r'Sum of squares:  $E(\mathbf{w})=\frac{1}{2}\sum_n(y_n-t_n)^2$ 
'); ax.set_xlabel('x'); ax.
set_ylabel('target'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula06_sum_squares
.png', dpi=180); plt.close(fig)
```



The dashed segments are residuals; squaring makes large mistakes count much more strongly.

notebook: `linear-models-for-regression.ipynb`

Notebook reference: least-squares objective.

Formula 7: Normal Equations

Formula 7 — The Normal Equations

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

“The best weights are found by a specific formula involving the input matrix and the target vector. Just plug in and compute!”

$\mathbf{w}^*$	the optimal weight vector
Φ	the design matrix , size $N \times (D + 1)$
Φ^T	transpose of Φ (rows become columns)
$(\Phi^T \Phi)^{-1}$	matrix inverse of the square matrix $\Phi^T \Phi$
$\mathbf{t}$	the vector of all training targets

Design Matrix — What Is It?

Φ is just a table: **one row per training example, one column per feature.**

Tiny example (try it by hand!):

3 training examples, 1 feature (x) plus a bias term.

$$\Phi = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{pmatrix} \quad \mathbf{t} = \begin{pmatrix} t_1 \\ t_2 \\ t_3 \end{pmatrix}$$

If $x = (0, 1, 2)$ and $t = (1, 3, 5)$:

$$\Phi = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} \quad \Phi^T \Phi = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix} \quad \Phi^T \mathbf{t} = \begin{pmatrix} 9 \\ 13 \end{pmatrix}$$

Solving gives $w_0 = 1$, $w_1 = 2$. The line is $y = 1 + 2x$.

notebook: `linear-models-for-regression.ipynb`

Normal Equations — Why It Matters

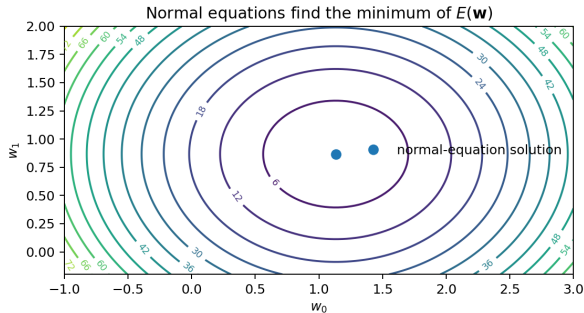
Why this matters: This is the **closed-form solution** of linear regression. No iterative training needed — just one formula.

- Predicting house prices, stock returns, temperature.
- Real-world: often the baseline against which all fancier methods are measured.
- Underlies the linear layer at the top of every neural network.

Formula 7 – Normal Equations: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(6)
x = np.linspace(-2,2,24); t = 1.0 + .9*x + rng.normal(0,.4,
    len(x))
ws0=np.linspace(-1,3,120); ws1=np.linspace(-.2,2,120); W0,W1=
    np.meshgrid(ws0,ws1)
E=np.empty_like(W0)
for i in range(W0.shape[0]):
    for j in range(W0.shape[1]): E[i,j]=.5*np.sum((W0[i,j]+W1
        [i,j]*x-t)**2)
X=np.column_stack([np.ones_like(x),x]); w=np.linalg.solve(X.
    T@X,X.T@t)
fig, ax = plt.subplots(figsize=(6.4,3.6)); cs=ax.contour(W0,
    W1,E,levels=12); ax.clabel(cs,inline=True,fontsize=7)
ax.plot(w[0],w[1], 'o', markersize=7, label='normal-equation
    solution'); ax.set_xlabel('$w_0$'); ax.set_ylabel('$w_1$')
ax.set_title(r'Normal equations find the minimum of $E(\mathbf{w})$'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/
    formula07_normal_equations.png', dpi=180); plt.close(
    fig)
```



The normal equations locate the weight vector at the bottom of the squared-error surface.

notebook: `linear-models-for-regression.ipynb`

Notebook reference: closed-form least-squares solution.

Formula 8: The Logistic Sigmoid

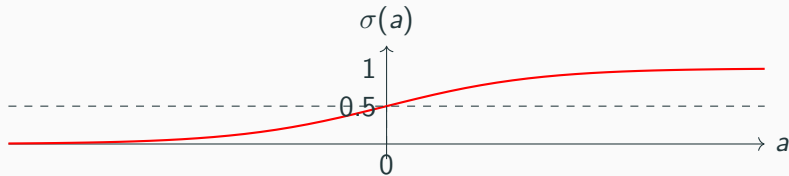
Formula 8 — The Logistic Sigmoid

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

“Squash any real number a into a number between 0 and 1. Perfect for turning a raw score into a probability.”

- a any real number ($-\infty$ to $+\infty$)
- $\sigma(a)$ the squashed value (between 0 and 1)
- e Euler's number, ≈ 2.718
- e^{-a} exponential of $-a$

The Sigmoid — A Picture



- At $a = 0$: $\sigma = 0.5$.
- Very negative a : $\sigma \rightarrow 0$.
- Very positive a : $\sigma \rightarrow 1$.

Sigmoid — Toy Example

Tiny example (try it by hand!):

- $a = 0$: $\sigma = 1/(1 + 1) = 0.5$.
- $a = 1$: $\sigma = 1/(1 + e^{-1}) = 1/(1 + 0.37) = 0.73$.
- $a = -1$: $\sigma = 1/(1 + e^1) = 1/(1 + 2.72) = 0.27$.
- $a = 5$: $\sigma \approx 0.993$ (almost certain).
- $a = -5$: $\sigma \approx 0.007$ (almost impossible).

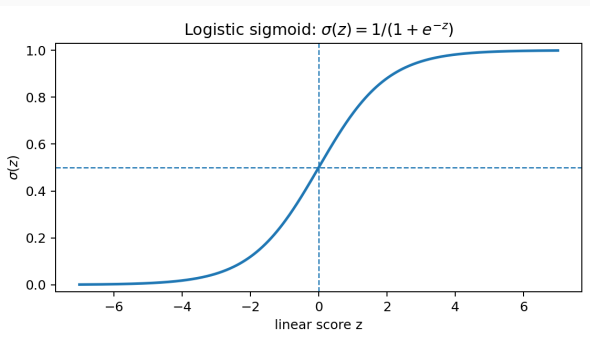
The sigmoid is the “conveyor belt” between scores and probabilities.

notebook: `logistic-regression.ipynb`

Formula 8 – The Logistic Sigmoid: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(-7,7,500); s=1/(1+np.exp(-x))
fig, ax=plt.subplots(figsize=(6.4,3.6)); ax.plot(x,s,
        linewidth=2); ax.axhline(.5,linestyle='--',linewidth
        =1); ax.axvline(0,linestyle='--',linewidth=1)
ax.set_ylim(-.03,1.03); ax.set_xlabel('linear score z'); ax.
set_ylabel(r'$\sigma(z)$'); ax.set_title(r'Logistic
sigmoid: $\sigma(z)=1/(1+e^{-z})$')
fig.tight_layout(); fig.savefig('./fig/
formula08_logistic_sigmoid.png', dpi=180); plt.close(
fig)
```



The sigmoid maps any real-valued score into a number between 0 and 1, making it usable as a probability.

notebook: `logistic-regression.ipynb`

Notebook reference: sigmoid / logistic model.

Formula 9: Logistic Regression

Formula 9 — Logistic Regression

$$p(\text{class 1} \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$$

“Compute a weighted sum of the inputs (like linear regression), then squash it with the sigmoid to get a probability of class 1.”

$p(\text{class 1} \mid \mathbf{x})$	probability that the input is class 1
$\mathbf{x}$	input vector
$\mathbf{w}$	weight vector (what we learn)
$\mathbf{w}^T \mathbf{x}$	weighted sum (a single number a)
σ	the sigmoid function from Formula 8

Logistic Regression — Toy Example (Spam Filter)

Tiny example (try it by hand!):

Two features:

- $x_1 = 1$ if the email contains “free” (else 0)
- $x_2 = 1$ if the email contains “money” (else 0)

Learned weights: $w_0 = -3$, $w_1 = 2$, $w_2 = 2$.

Email A: has “free” but not “money”.

$$a = -3 + 2(1) + 2(0) = -1 \quad \Rightarrow \quad p(\text{spam}) = \sigma(-1) \approx 0.27$$

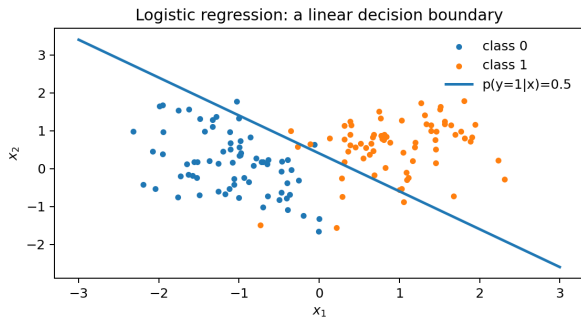
Email B: has both words.

$$a = -3 + 2 + 2 = 1 \quad \Rightarrow \quad p(\text{spam}) = \sigma(1) \approx 0.73$$

Formula 9 – Logistic Regression: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(9)
a=rng.normal([-1,0],[.65,.7],size=(70,2)); b=rng.normal
([1,.5],[.65,.7],size=(70,2))
xx=np.linspace(-3,3,200); boundary=.4-xx
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.scatter(a[:,0],a
[:,1],s=12,label='class 0'); ax.scatter(b[:,0],b
[:,1],s=12,label='class 1'); ax.plot(xx,boundary,
linewidth=2,label='p(y=1|x)=0.5')
ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$'); ax.set_title(
'Logistic regression: a linear decision boundary');
ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/
formula09_logistic_regression.png', dpi=180); plt.
close(fig)
```



Logistic regression learns a linear score and turns that score into a class probability.

notebook: `logistic-regression.ipynb`

Notebook reference: binary logistic regression.

Formula 10: Cross-Entropy Loss

Formula 10 — Cross-Entropy Loss

$$E(\mathbf{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

“For every example, look at the probability we gave to the **correct** answer, take its logarithm, and add up the negatives. We want to make this small — meaning we gave high probability to the right answer.”

t_n true label: 1 or 0

y_n predicted probability of class 1

$\ln$ natural logarithm

N number of examples

Cross-Entropy — Toy Example

Tiny example (try it by hand!):

Example 1: true label $t = 1$, prediction $y = 0.9$.

Loss $= -\ln(0.9) = 0.105$ (small — good prediction).

Example 2: true label $t = 1$, prediction $y = 0.1$.

Loss $= -\ln(0.1) = 2.30$ (large — bad prediction!).

Example 3: true label $t = 0$, prediction $y = 0.2$.

Loss $= -\ln(0.8) = 0.223$ (small).

Total loss $= 0.105 + 2.30 + 0.223 \approx 2.63$.

Cross-entropy is **how surprised** we are at the truth, given our predictions. Good predictions $\Rightarrow$ small surprise.

notebook: `logistic-regression.ipynb`

Cross-Entropy — Step-by-Step Calculation (1/2)

Recall the formula:

$$E = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

Example 1: $t_1 = 1, y_1 = 0.9$.

$$t_1 \ln y_1 = 1 \times \ln(0.9) = -0.1054$$

$$(1 - t_1) \ln(1 - y_1) = 0 \times \ln(0.1) = 0$$

$$\text{Contribution} = -(-0.1054 + 0) = +0.1054$$

Example 2: $t_2 = 1, y_2 = 0.1$.

$$t_2 \ln y_2 = 1 \times \ln(0.1) = -2.3026$$

$$(1 - t_2) \ln(1 - y_2) = 0 \times \ln(0.9) = 0$$

$$\text{Contribution} = -(-2.3026 + 0) = +2.3026$$

Tiny example (try it by hand!):

Notice: only **one** of the two terms is nonzero per example!

Cross-Entropy — Step-by-Step Calculation (2/2)

Example 3: $t_3 = 0$, $y_3 = 0.2$.

$$t_3 \ln y_3 = 0 \times \ln(0.2) = 0$$

$$(1 - t_3) \ln(1 - y_3) = 1 \times \ln(0.8) = -0.2231$$

$$\text{Contribution} = -(0 + -0.2231) = +0.2231$$

Total loss:

$$E = 0.1054 + 2.3026 + 0.2231 = 2.6311$$

Tiny example (try it by hand!):

Why the logarithm?

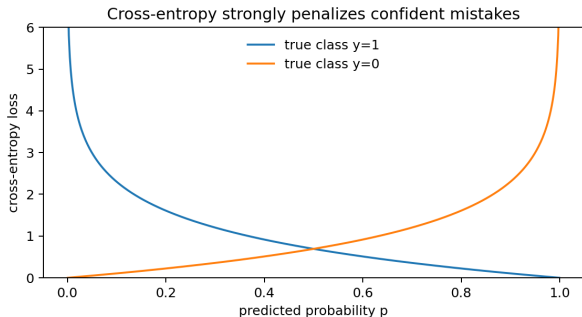
- $\ln(1) = 0$: perfect prediction gives no penalty.
- $\ln(0) = -\infty$: catastrophic prediction gets infinite penalty.
- It is the negative log-likelihood of the data, so minimising it maximises the chance of the correct label.

Cross-entropy punishes the model in proportion to how **surprised** it would be by the truth.

Formula 10 – Cross-Entropy Loss: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
p=np.linspace(.001,.999,500); loss1=-np.log(p); loss0=-np.log(1-p)
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.plot(p,loss1,label='true class y=1'); ax.plot(p,loss0,label='true class y=0')
ax.set_ylim(0,6); ax.set_xlabel('predicted probability p'); ax.set_ylabel('cross-entropy loss'); ax.set_title(r'Cross-entropy strongly penalizes confident mistakes'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula10_cross_entropy.png', dpi=180); plt.close(fig)
```



Confident wrong predictions receive a large penalty, which makes cross-entropy useful for classification training.

notebook: `logistic-regression.ipynb`

Notebook reference: *likelihood / log-loss view.*

Formula 11: The Kernel Function

Formula 11 — The Kernel Function

$$k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}')$$

“A kernel is a **similarity measure** between two inputs. It secretly computes a high-dimensional dot product without ever going to that high dimension.”

$k(\mathbf{x}, \mathbf{x}')$	similarity score between inputs $\mathbf{x}$ and $\mathbf{x}'$
$\phi(\mathbf{x})$	some (possibly very high-dimensional) feature map
$\phi(\mathbf{x})^T \phi(\mathbf{x}')$	dot product in that feature space

Kernel — Common Choices

Linear kernel

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$$

Just the plain dot product.

Gaussian (RBF) kernel

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

Similar if close, dissimilar if far.

Polynomial kernel

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^M$$

Captures feature interactions up to degree M .

notebook: `kernel-regression.ipynb`

Kernel — Toy Example

Tiny example (try it by hand!):

Two points: $\mathbf{x} = (1, 0)$, $\mathbf{x}' = (0, 1)$.

Linear kernel:

$$k = 1 \cdot 0 + 0 \cdot 1 = 0 \quad (\text{orthogonal — no similarity}).$$

Gaussian kernel with $\sigma = 1$:

$$\|\mathbf{x} - \mathbf{x}'\|^2 = (1 - 0)^2 + (0 - 1)^2 = 2 \quad \Rightarrow \quad k = e^{-2/2} = e^{-1} \approx 0.37$$

Not zero — the Gaussian kernel sees them as “somewhat similar.”

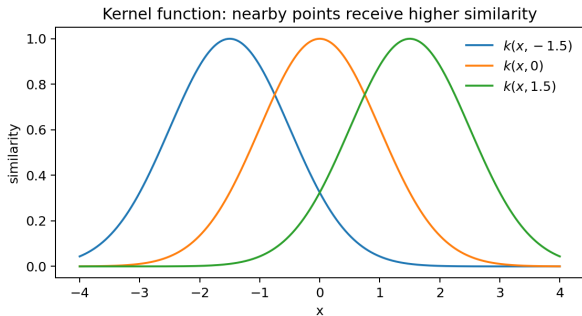
Different kernels answer the question “how similar are these two inputs?” differently. Choose the one that matches your problem.

notebook: `kernel-regression.ipynb`

Formula 11 – The Kernel Function: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(-4,4,500); centers=[-1.5,0,1.5]
fig,ax=plt.subplots(figsize=(6.4,3.6))
for c in centers: ax.plot(x,np.exp(-.5*(x-c)**2),label=rf'$k(x, {c:g})$')
ax.set_xlabel('x'); ax.set_ylabel('similarity'); ax.set_title(
    'Kernel function: nearby points receive higher similarity'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('./fig/formula11_kernel.png',
    , dpi=180); plt.close(fig)
```



A kernel measures similarity: nearby points receive high weight, while distant points receive little weight.

notebook: kernel-regression.ipynb

Notebook reference: Gaussian kernel regression.

Formula 12: The SVM Margin

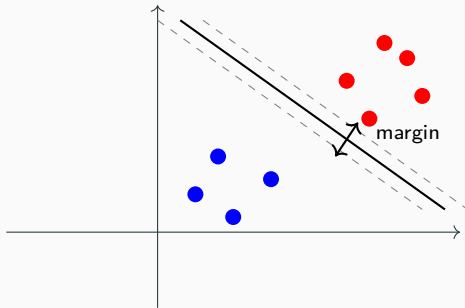
Formula 12 — The SVM Margin

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1$$

“Find the line (or hyperplane) that separates the two classes with the **widest possible margin** — the biggest safety buffer around the decision boundary.”

$\mathbf{w}, b$	weights and bias of the line
$\ \mathbf{w}\ ^2$	length of $\mathbf{w}$ squared
t_n	label: +1 or -1
$\mathbf{x}_n$	the n -th training input
$\mathbf{w}^T \mathbf{x}_n + b$	signed distance from the line
1	the margin is $2/\ \mathbf{w}\ $

SVM — A Picture



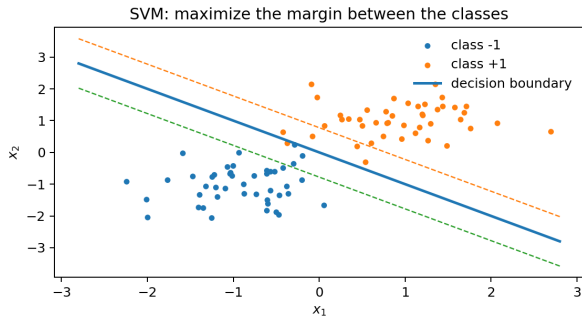
- Support vectors: the **closest** points to the boundary.
- Only support vectors matter — other points don't affect the line.
- Wider margin $\Rightarrow$ better generalisation.

notebook: `support-vector-machines.ipynb`

Formula 12 – The SVM Margin: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(12); a=rng.normal
    ([-1,-1],[.65,.55],size=(45,2)); b=rng.normal
    ([1,1],[.65,.55],size=(45,2))
xx=np.linspace(-2.8,2.8,300); mid=-xx; m=.55*np.sqrt(2)
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.scatter(a[:,0],a
[:,1],s=11,label='class -1'); ax.scatter(b[:,0],b
[:,1],s=11,label='class +1'); ax.plot(xx,mid,
linewidth=2,label='decision boundary'); ax.plot(xx,
mid+m,'--',linewidth=1); ax.plot(xx,mid-m,'--',
linewidth=1)
ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$'); ax.set_title(
'SVM: maximize the margin between the classes'); ax.
legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula12_svm_margin.
png', dpi=180); plt.close(fig)
```



The separator is chosen to leave the largest possible gap between the two classes.

notebook: `support-vector-machines.ipynb`

Notebook reference: maximum-margin classification.

Formula 13: Gaussian Process Prediction

Formula 13 — Gaussian Process Prediction

$$m(\mathbf{x}_\star) = \mathbf{k}_\star^T \mathbf{C}^{-1} \mathbf{t} \quad \sigma^2(\mathbf{x}_\star) = c_\star - \mathbf{k}_\star^T \mathbf{C}^{-1} \mathbf{k}_\star$$

“The predicted mean is a similarity-weighted average of the training targets. The uncertainty is what remains after subtracting the information we have about the new point.”

$\mathbf{x}_\star$	a new test input
$\mathbf{k}_\star$	similarities between $\mathbf{x}_\star$ and each training point
$\mathbf{C}$	covariance matrix of training targets (with noise)
$\mathbf{t}$	training target vector
$c_\star$	$k(\mathbf{x}_\star, \mathbf{x}_\star) + \beta^{-1}$
$\sigma^2(\mathbf{x}_\star)$	predictive uncertainty at $\mathbf{x}_\star$

Gaussian Process — Why It's Beautiful

A Gaussian process is a **distribution over functions**. Instead of guessing *parameters* (weights), we guess the *whole curve* at once, with uncertainty.

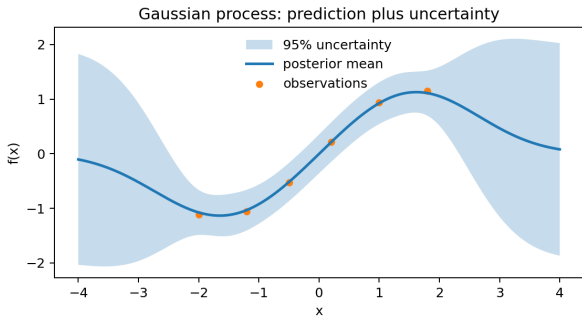
- Small data? GPs are excellent.
- Uncertain far from data? The variance grows.
- Confident near data? The variance shrinks.
- Use it for: hyperparameter tuning (Bayesian optimisation), sensor calibration, geostatistics (kriging).

notebook: `gaussian-processes.ipynb`

Formula 13 – Gaussian Process Prediction: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
X=np.array([-2,-1.2,-.5,.2,1.0,1.8]); y=np.sin(X)+.1*X
x=np.linspace(-4,4,350)
def k(a,b,l=1.0): return np.exp(-.5*((a[:None]-b[None,:])/l)**2)
K=k(X,X)+.05*np.eye(len(X)); Ks=k(x,X); mean=Ks@np.linalg.solve(K,y); var=1-np.sum(Ks*np.linalg.solve(K,Ks.T).T,axis=1); sd=np.sqrt(np.maximum(var,0))
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.fill_between(x,mean-1.96*sd,mean+1.96*sd,alpha=.25,label='95% uncertainty'); ax.plot(x,mean,linewidth=2,label='posterior mean'); ax.scatter(X,y,s=20,label='observations')
ax.set_title('Gaussian process: prediction plus uncertainty')
; ax.set_xlabel('x'); ax.set_ylabel('f(x)'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula13_gaussian_process.png', dpi=180); plt.close(fig)
```



A Gaussian process gives both a central prediction and a principled uncertainty band.

notebook: `gaussian-processes.ipynb`

Notebook reference: *GP predictive mean and variance.*

Formula 14: The Gaussian Mixture

Formula 14 — Gaussian Mixture

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

“Add up several bell curves, each with its own centre, spread, and weight. The result can have **multiple peaks** — much more flexible than one bell.”

K	number of Gaussians we mix
π_k	mixing weight (how important component k is)
$\boldsymbol{\mu}_k$	mean of component k
$\boldsymbol{\Sigma}_k$	covariance of component k
$\sum_k \pi_k = 1$	weights must sum to 1

Gaussian Mixture — Toy Example

Tiny example (try it by hand!):

Heights of adults: men average 178 cm, women average 165 cm. If the population is 50% men, 50% women:

$$p(\text{height}) = 0.5 \mathcal{N}(\text{height} \mid 178, 8^2) + 0.5 \mathcal{N}(\text{height} \mid 165, 6^2)$$

The result is a **bimodal** distribution (two peaks). A single Gaussian cannot capture this shape!

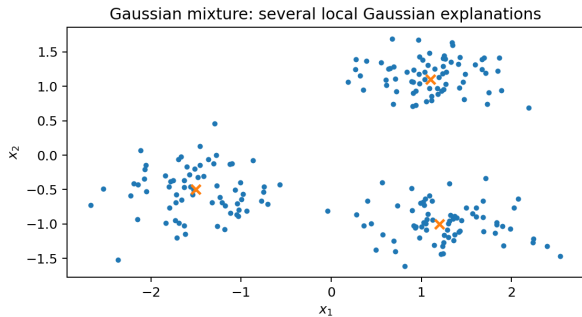
Mixture models are how we cluster data. Each bell curve is a **cluster**. Points are softly assigned to clusters.

notebook: `gaussian-mixture-models.ipynb`

Formula 14 – The Gaussian Mixture: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(14); m=[(-1.5,-.5),(1.1,1.1),
    (1.2,-1.0)]; c=[(.55,.35),(.45,.25),(.5,.3)]
X=np.vstack([rng.normal(loc,scale,(70,2)) for loc,scale in
    zip(m,c)])
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.scatter(X[:,0],X
   [:,1],s=9); ax.scatter([q[0] for q in m],[q[1] for q
    in m],s=50,marker='x',linewidths=2)
ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$'); ax.set_title(
    'Gaussian mixture: several local Gaussian
    explanations')
fig.tight_layout(); fig.savefig('./fig/
    formula14_gaussian_mixture.png', dpi=180); plt.close(
    fig)
```



Several Gaussian components can explain different regions of the data, with soft rather than hard assignments.

notebook: gaussian-mixture-models.ipynb

Notebook reference: mixture of Gaussians.

Formula 15: KL Divergence

Formula 15 — KL Divergence

$$\text{KL}(p \parallel q) = \int p(x) \ln \frac{p(x)}{q(x)} dx$$

“A measure of how different distribution q is from distribution p . It is always ≥ 0 . Zero means they are identical.”

$p(x)$ the “true” distribution

$q(x)$ our approximation

$\ln$ natural logarithm

$\int$ integral over all x

$\text{KL}(p \parallel q)$ the divergence (a non-negative number)

KL Divergence — Why It Matters

Why this matters: KL divergence is **the** distance measure for probability distributions. It appears in nearly every modern ML algorithm.

- **Variational inference:** choose q closest to true p .
- **Variational autoencoders (VAEs):** loss function contains KL.
- **Expectation propagation:** minimises KL in reverse.
- **Model comparison:** how different are two candidate models?

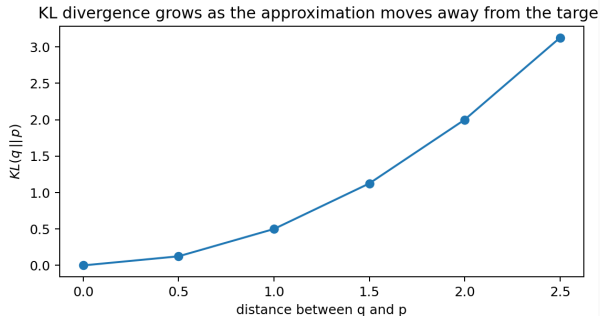
KL is **not symmetric** — $KL(p||q) \neq KL(q||p)$. That is why VAEs and GANs behave so differently!

`notebook: variational-univariate-gaussian.ipynb`

Formula 15 – KL Divergence: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(-4,4,500); p=np.exp(-.5*x*x)/np.sqrt(2*np.pi);
shifts=np.linspace(0,.0+2.5,6)
kl=[0.5*s*s for s in shifts]
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.plot(shifts,kl,
marker='o'); ax.set_xlabel('distance between q and p'
); ax.set_ylabel(r'$KL(q||p)$'); ax.set_title('KL
divergence grows as the approximation moves away
from the target')
fig.tight_layout(); fig.savefig('../fig/
formula15_kl_divergence.png', dpi=180); plt.close(fig
)
```



KL divergence is zero only when the approximation matches the target; otherwise it measures the mismatch.

notebook: `variational-univariate-gaussian.ipynb`

Notebook reference: variational approximation and KL objective.

Formula 16: The EM Algorithm

Formula 16 — The EM Algorithm

$$\ln p(\mathbf{X} \mid \boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q \parallel p)$$

E step: set $q(\mathbf{Z}) = p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$.

M step: maximise $\mathbb{E}[\ln p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})]$ with respect to $\boldsymbol{\theta}$.

“Guess the hidden variables (E step), then update the parameters assuming the guesses are right (M step). Repeat until stable.”

- $\mathbf{X}$ observed data
- $\mathbf{Z}$ hidden (latent) variables
- $\boldsymbol{\theta}$ model parameters
- $\mathcal{L}$ lower bound on the log-likelihood
- KL KL divergence from Formula 15

EM — Toy Example (Two Coin Problem)

Tiny example (try it by hand!):

Two coins A and B . You flip a mystery coin 5 times, see H, T, H, H, T , then flip another mystery coin 5 times, see H, H, H, T, H . You do NOT know which flips came from which coin.

E step: guess probabilities, e.g. $p_A = 0.6$, $p_B = 0.5$.

For each flip-sequence, compute probability under each coin.

M step: re-estimate p_A, p_B as weighted averages using the guesses.

Repeat. After a few iterations, EM locks onto the true probabilities.

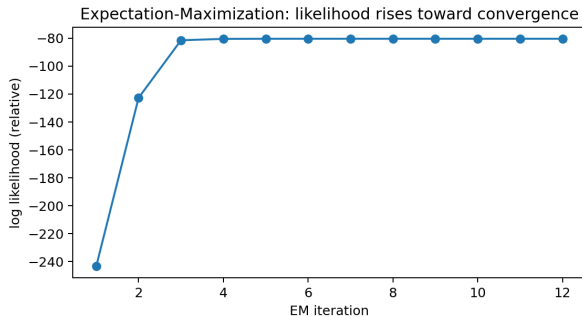
EM is **how you train models with hidden variables** — mixtures, HMMs, latent Dirichlet allocation (LDA), missing-data models.

notebook: `gaussian-mixture-models.ipynb`

Formula 16 – The EM Algorithm: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(16); x=np.r_[rng.normal(-1,.45,90),
    rng.normal(1,.55,90)]
means=np.array([-0.2,.2]); ll=[]
for _ in range(12):
    r1=np.exp(-.5*(x-means[0])**2/.45**2); r2=np.exp(-.5*(x-
    means[1])**2/.55**2); r=r1/(r1+r2)
    means=np.array([(r*x).sum()/r.sum(),((1-r)*x).sum()/(1-r)
    .sum()]); ll.append(np.log(r1+r2).sum())
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.plot(range(1,len(
    ll)+1),ll,marker='o'); ax.set_xlabel('EM iteration');
    ax.set_ylabel('log likelihood (relative)'); ax.
    set_title('Expectation-Maximization: likelihood rises
    toward convergence')
fig.tight_layout(); fig.savefig('./fig/formula16_em.png',
    dpi=180); plt.close(fig)
```



EM alternates between estimating hidden responsibilities and updating parameters; the likelihood typically rises toward convergence.

notebook: `gaussian-mixture-models.ipynb`

Formula 17: Principal Component Analysis

Formula 17 — PCA (The Eigenvector Equation)

$$\mathbf{S} \mathbf{u}_i = \lambda_i \mathbf{u}_i$$

“The directions of highest variance in the data are the **eigenvectors** of the covariance matrix. Each direction has a corresponding variance — the **eigenvalue**.”

S data covariance matrix (how features co-vary)

$\mathbf{u}_i$ an eigenvector (a direction in feature space)

λ_i the eigenvalue (variance along that direction)

PCA — What It Really Does

Imagine photographing a 3D object. The best angle is the one that shows the most detail. PCA finds those best angles — the directions along which the data varies most.

- Sort eigenvectors by eigenvalue (largest variance first).
- Keep the top M directions as your new “features.”
- Discard the rest (they carry little information).

Tiny example (try it by hand!):

Data: 100-dimensional. Eigenvalues: $(10, 5, 0.1, 0.05, \dots)$.

The first two directions explain 99% of the variance.

Compress: 100 numbers $\rightarrow$ 2 numbers. Huge savings!

notebook: `principal-component-analysis.ipynb`

PCA — How the 99% Is Computed (1/2)

Step 1: fraction explained by each component.

$$\text{Fraction}_i = \frac{\lambda_i}{\lambda_1 + \lambda_2 + \cdots + \lambda_D}$$

Given eigenvalues: $\lambda_1 = 10$, $\lambda_2 = 4.8$, $\lambda_3 = 0.1$, $\lambda_4 = 0.05$, remaining $\lambda_5 + \cdots + \lambda_{100} = 0.05$.

Sum of all eigenvalues:

$$\sum_{j=1}^{100} \lambda_j = 10 + 4.8 + 0.1 + 0.05 + 0.05 = 15.0$$

The total variance of the data is 15.0 (in whatever units our features happen to be).

PCA — How the 99% Is Computed (2/2)

Step 2: individual contributions.

$$\lambda_1/\text{total} = 10.00/15.0 = 66.7\%$$

$$\lambda_2/\text{total} = 4.80/15.0 = 32.0\%$$

$$\lambda_3/\text{total} = 0.10/15.0 = 0.67\%$$

$$\lambda_4/\text{total} = 0.05/15.0 = 0.33\%$$

$$\text{all others} = 0.05/15.0 = 0.33\%$$

Cumulative:

$$\text{First 1} = 66.7\%, \quad \text{First 2} = 66.7 + 32.0 = 98.7\% \approx 99\%$$

Tiny example (try it by hand!):

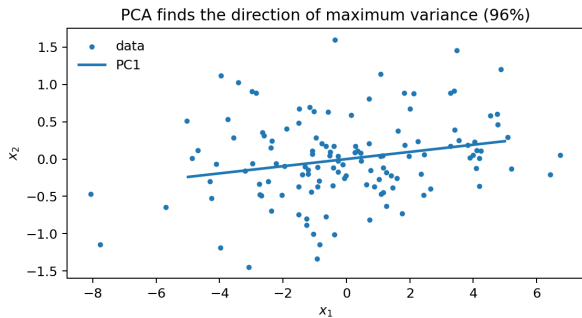
So the first two directions capture almost all the information. We can compress 100 numbers down to 2 per data point and keep about 99% of the signal.

In real data, eigenvalues drop off more slowly. **Always inspect your own eigenvalue spectrum** before deciding how many components to keep.

Formula 17 – Principal Component Analysis: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(17); z=rng.normal(size=(130,2)); A=
    np.array([[2.8,1.0],[0,.55]]); X=z@A.T; X-=X.mean(0)
vals,vecs=np.linalg.eigh(np.cov(X,rowvar=False)); u=vecs
[:, -1]; t=np.linspace(-5,5,100)
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.scatter(X[:,0],X
[:,1],s=9,label='data'); ax.plot(t*u[0],t*u[1],
    linewidth=2,label='PC1')
ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$'); ax.set_title(
    f'PCA finds the direction of maximum variance ({vals
    [-1]/vals.sum():.0%})'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('./fig/formula17_pca.png',
    dpi=180); plt.close(fig)
```



PCA rotates the coordinate system so the first principal component captures the largest variance.

notebook: `principal-component-analysis.ipynb`

Notebook reference: eigenvectors and variance explained.

Formula 18: The HMM Joint Distribution

Formula 18 — The Hidden Markov Model

$$p(\mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{z}_1, \dots, \mathbf{z}_N) = p(\mathbf{z}_1) \prod_{n=2}^N p(\mathbf{z}_n \mid \mathbf{z}_{n-1}) \prod_{n=1}^N p(\mathbf{x}_n \mid \mathbf{z}_n)$$

“At each step n , the world is in some hidden state $\mathbf{z}_n$ (which we cannot see). We observe $\mathbf{x}_n$ (which depends only on the current hidden state). The hidden state evolves over time.”

N	length of the sequence
$\mathbf{x}_n$	observed value at time n
$\mathbf{z}_n$	hidden (latent) state at time n
$p(\mathbf{z}_n \mid \mathbf{z}_{n-1})$	transition (how states evolve)
$p(\mathbf{x}_n \mid \mathbf{z}_n)$	emission (how states produce observations)

HMM — Toy Example (Weather and Umbrella)

Tiny example (try it by hand!):

Hidden states: **Sunny, Rainy**.

Observed: whether your neighbour carries an **umbrella**.

Transition: Sunny $\rightarrow$ Sunny 80%, Sunny $\rightarrow$ Rainy 20%, etc.

Emission: Rainy $\rightarrow$ Umbrella 90%, Sunny $\rightarrow$ Umbrella 10%.

You observe: umbrella, no-umbrella, umbrella, umbrella.

What is the most likely weather sequence? That's what the HMM tells us!

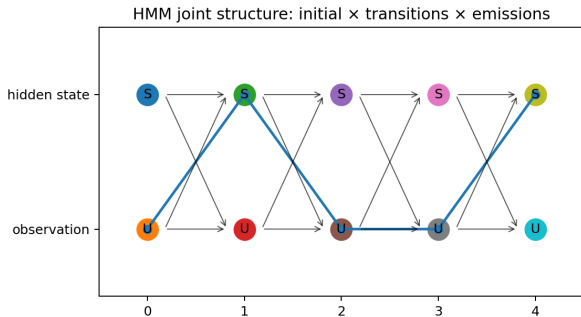
You only see the **symptoms**, but you want the **cause**. HMMs do this reverse reasoning using Bayes' theorem over time.

notebook: `hidden-markov-model.ipynb`

Formula 18 – The HMM Joint Distribution: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
T=5; ys=np.array([0,1,0,0,1]); xs=np.arange(T)
fig,ax=plt.subplots(figsize=(6.4,3.6));
for t in xs:
    ax.scatter(t,1, s=260); ax.scatter(t,0,s=260)
    ax.text(t,1,'S',ha='center',va='center'); ax.text(t,0,'U'
            ,ha='center',va='center')
    if t<T-1:
        for y0 in [0,1]:
            for y1 in [0,1]: ax.annotate('',xy=(t+0.82,y1),
                xytext=(t+0.18,y0),arrowprops=dict(arrowstyle='->',lw
                    =.8,alpha=.65))
ax.plot(xs,ys,linewidth=2,marker='o'); ax.set_xlim(-.5,T-.5);
ax.set_ylim(-.5,1.5); ax.set_xticks(xs); ax.
set_yticks([0,1],['observation','hidden state']); ax.
set_title('HMM joint structure: initial x transitions x
    emissions')
fig.tight_layout(); fig.savefig('../fig/formula18_hmm_joint.
png', dpi=180); plt.close(fig)
```



The joint probability factors into an initial state term, transition terms, and emission terms.

notebook: `hidden-markov-model.ipynb`

Notebook reference: hidden-state sequence model.

Formula 19: The Forward Algorithm

Formula 19 — The Forward Algorithm

$$\alpha(\mathbf{z}_n) = p(\mathbf{x}_n \mid \mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} \alpha(\mathbf{z}_{n-1}) p(\mathbf{z}_n \mid \mathbf{z}_{n-1})$$

“How likely is it that we are in state $\mathbf{z}_n$ after seeing the first n observations? Multiply: (probability of previous state) $\times$ (transition) summed over all previous states, then times (how likely we are to see today's observation).”

$\alpha(\mathbf{z}_n)$	forward message: probability of state n given data so far
$p(\mathbf{x}_n \mid \mathbf{z}_n)$	emission probability at time n
$p(\mathbf{z}_n \mid \mathbf{z}_{n-1})$	transition probability
$\sum_{\mathbf{z}_{n-1}}$	sum over all possible previous states

Forward Algorithm — Why It's Efficient

A sequence of length N with K states has K^N possible paths. Naively checking every path is **exponential**. The forward algorithm does it in $N \times K^2$ steps — **linear**.

Tiny example (try it by hand!):

Speech recognition: $N = 100$ frames, $K = 50$ phonemes.

Naive: 50^{100} paths (impossible!).

Forward algorithm: $100 \times 50^2 = 250,000$ operations — easy!

Why this matters: This is why HMMs work on real data. The forward–backward algorithm is a beautiful example of exploiting structure to make the impossible possible.

notebook: `hidden-markov-model.ipynb`

Forward Algorithm — Step-by-Step Calculation (1/3)

Setup. 2 hidden states (Sunny, Rainy), 2 observations (U = umbrella, N = no umbrella).

- Initial: $\pi(\text{Sunny}) = 0.6$, $\pi(\text{Rainy}) = 0.4$.
- Emissions: $p(U|\text{Sunny}) = 0.1$, $p(U|\text{Rainy}) = 0.9$. Similarly $p(N|\text{Sunny}) = 0.9$, $p(N|\text{Rainy}) = 0.1$.
- Transitions: $S \rightarrow S = 0.8$, $S \rightarrow R = 0.2$, $R \rightarrow S = 0.3$, $R \rightarrow R = 0.7$.

Step 1 ($n = 1$). Observed $x_1 = U$.

$$\alpha(z_1) = p(x_1|z_1) \pi(z_1)$$

$$\alpha(\text{Sunny}_1) = 0.1 \times 0.6 = 0.06$$

$$\alpha(\text{Rainy}_1) = 0.9 \times 0.4 = 0.36$$

After one observation (umbrella), Rainy is $6\times$ more likely than Sunny.

Forward Algorithm — Step-by-Step Calculation (2/3)

Step 2 ($n = 2$). Observed $x_2 = N$.

$$\alpha(z_2) = p(x_2|z_2) \sum_{z_1} \alpha(z_1) p(z_2|z_1)$$

For Sunny at $n = 2$:

$$\sum_{z_1} = 0.06 \times 0.8 + 0.36 \times 0.3 = 0.048 + 0.108 = 0.156$$

$$\alpha(\text{Sunny}_2) = 0.9 \times 0.156 = 0.1404$$

For Rainy at $n = 2$:

$$\sum_{z_1} = 0.06 \times 0.2 + 0.36 \times 0.7 = 0.012 + 0.252 = 0.264$$

$$\alpha(\text{Rainy}_2) = 0.1 \times 0.264 = 0.0264$$

Two observations are enough to shift the belief: the umbrella favoured Rainy, but the missing umbrella swung it back to Sunny.

Forward Algorithm — Step-by-Step Calculation (3/3)

Posterior at $n = 2$ (after both observations). Normalise the α values so they sum to 1:

$$Z = 0.1404 + 0.0264 = 0.1668$$
$$p(\text{Sunny}_2 \mid U, N) = \frac{0.1404}{0.1668} = 0.842 \quad p(\text{Rainy}_2 \mid U, N) = \frac{0.0264}{0.1668} = 0.158$$

- After U alone: Rainy was favoured.
- After U and N : Sunny is favoured.
- The forward algorithm tracks this shift in one pass.

The forward variable $\alpha(z_n)$ is the joint probability of the data so far **and** being in state z_n . It is the hidden quantity that lets us answer every question about the sequence.

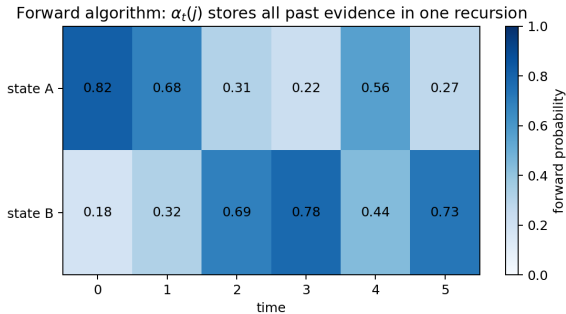
notebook: hidden-markov-model.ipynb

Formula 19 – The Forward Algorithm: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
alpha=np.array
    ([[.82,.18],[.68,.32],[.31,.69],[.22,.78],[.56,.44],[.27,.73]])

fig,ax=plt.subplots(figsize=(6.4,3.6)); im=ax.imshow(alpha.T,
    aspect='auto',vmin=0,vmax=1,cmap='Blues')
for i in range(2):
    for j in range(alpha.shape[0]): ax.text(j,i,f'{alpha[j,i]
        :.2f}',ha='center',va='center')
ax.set_xticks(range(alpha.shape[0])); ax.set_yticks([0,1,['
    state A','state B']]); ax.set_xlabel('time'); ax.
    set_title(r'Forward algorithm:  $\alpha_t(j)$  stores
    all past evidence in one recursion'); fig.colorbar(im
    ,ax=ax,label='forward probability')
fig.tight_layout(); fig.savefig('./fig/formula19_forward.png
    ', dpi=180); plt.close(fig)
```



The forward value $\alpha_t(j)$ compresses all observations seen so far into one recursive quantity.

notebook: `hidden-markov-model.ipynb`

Notebook reference: *forward-message recursion*.

Formula 20: The Kalman Filter

Formula 20 — The Kalman Filter

$$\mu_n = \mathbf{A}\mu_{n-1} + \mathbf{K}_n(x_n - \mathbf{C}\mathbf{A}\mu_{n-1})$$

“Predict where the state will be, compare to the measurement, then correct by a fraction of the error. The fraction is the **Kalman gain**.”

μ_n	our estimate of the hidden state at time n
$\mathbf{A}$	dynamics matrix (how state evolves)
$\mathbf{C}$	measurement matrix (how state produces observations)
x_n	the noisy measurement at time n
$\mathbf{K}_n$	Kalman gain (how much to trust the new measurement)
$\mathbf{C}\mathbf{A}\mu_{n-1}$	our predicted observation

Kalman Filter — Toy Example (Tracking a Car)

Tiny example (try it by hand!):

A car moves along a straight road. You get GPS readings every second (noisy). You know the car's speed limit but not its exact speed.

Prediction: use physics — position next second $\approx$ position now + velocity $\times$ 1s.

Correction: GPS says the car is here. Take a weighted average of prediction and GPS, weighted by how much you trust each.

If GPS is very accurate, weight it more (large **K**).

If GPS is noisy, weight your prediction more (small **K**).

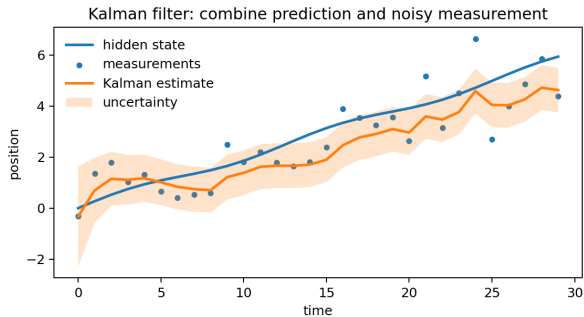
Why this matters: Kalman filters landed on the Moon (Apollo guidance), and today they run inside every phone's GPS, every drone's autopilot, and every autonomous car.

notebook: `linear-dynamical-system.ipynb`

Formula 20 – The Kalman Filter: Python Example

Core code

```
import numpy as np
import matplotlib.pyplot as plt
rng=np.random.default_rng(20); t=np.arange(30); true=.2*t +
.15*np.sin(t/2); obs=true+rng.normal(0,.9,len(t));
est=np.zeros(len(t)); P=np.zeros(len(t)); est[0]=obs
[0]; P[0]=1
Q,R=.08,.7
for k in range(1,len(t)):
    pred=est[k-1]; Ppred=P[k-1]+Q; K=Ppred/(Ppred+R); est[k]=
    pred+K*(obs[k]-pred); P[k]=(1-K)*Ppred
s=1.96*np.sqrt(P)
fig,ax=plt.subplots(figsize=(6.4,3.6)); ax.plot(t,true,
linewidth=2,label='hidden state'); ax.scatter(t,obs,s
=12,label='measurements'); ax.plot(t,est,linewidth=2,
label='Kalman estimate'); ax.fill_between(t,est-s,est
+s,alpha=.22,label='uncertainty')
ax.set_xlabel('time'); ax.set_ylabel('position'); ax.
set_title('Kalman filter: combine prediction and
noisy measurement'); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig('../fig/formula20_kalman.png',
dpi=180); plt.close(fig)
```



The filter alternates prediction and measurement update, balancing model uncertainty against noisy observations.

notebook: linear-dynamical-system.ipynb

The upstream notebook is conceptual only; this is a standalone teaching calculation.

Wrap-Up

The 20 Formulas at a Glance

1. Sum rule
2. Bayes' theorem
3. Gaussian
4. Expectation
5. Linear model
6. Squared error
7. Normal equations
8. Sigmoid
9. Logistic regression
10. Cross-entropy
11. Kernel function
12. SVM margin
13. Gaussian process
14. Gaussian mixture
15. KL divergence
16. EM algorithm
17. PCA
18. HMM joint
19. Forward algorithm
20. Kalman filter

What to Learn Next

- **If you liked the probability part:** Murphy, *Machine Learning: A Probabilistic Perspective*.
- **If you liked the deep-learning part:** Goodfellow, Bengio, Courville, *Deep Learning*.
- **If you liked the practical part:** run the notebooks from `gerdm/prml`.
- **Then come back** and read Bishop's PRML for full derivations.

Thank you! Questions?

References

-  C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
-  G. D. M. et al. gerdm/prml on GitHub. <https://github.com/gerdm/prml>
-  K. P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
-  I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.
-  T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*. Springer, 2009.